

Distributed System COMP90015

Junesuh Kang - 1239956

Problem Context

The assignment focuses on building a client-server system for managing university subject enrolments. A single server holds the list of subjects; any number of clients can connect to it concurrently, each acting as a university administrator with identical and unrestricted access. There is no login and no distinction between clients: any client may look up a subject's details, enrol a student, withdraw one, transfer a student between two subjects, or change a subject's capacity.

The problem occurs with the fact that a single mutable dataset is exposed to an arbitrary number of simultaneous writers (clients) across the network. Unsynchronised interleaving, the random context switching between the threads, corrupts state: 1) two admins enrolling into the last remaining capacity can both observe the seat as free before either takes it, and the subject ends up exceeding the maximum capacity. 2) An operation spanning two subjects (transfer) must appear instantaneous to every other client, or a student becomes observable as enrolled in both subjects at once, or in neither. 3) Coarse synchronisation through a single global lock eliminates race conditions but serialises the entire server, which can lead to unrelated subjects being blocked.

The system must also be durable: every successful write to a client must survive a process crash, and must be restored on restart from the server-managed file rather than the original input, where the contents may be stale.

Component Description

The program consists of two executables: *SubjectServer* and *SubjectClient*, communicating over **Transmission Control Protocol (TCP)** with one JSON object per line. The server is divided into four classes plus their nested helpers, separated into networking, durability and system logic. The most important section of the program design is that every lock in the server is acquired inside a single class, *SubjectRegistry*, which makes the concurrency control a complete and inspectable inventory in one file, and makes the deadlock avoidance and delay placement rules checkable by reading one page of code.

File: `SubjectServer.java`

Responsibility/Function: Validates the three command line arguments, loads and validates the subject file, opens the *ServerSocket*, runs the accept loop on its own thread, and hosts the console (status, quit) and the graceful shutdown sequence. Its nested Sessions class tracks live connections and in-flight operations under a single monitor.

File: `ClientHandler.java`

Responsibility/Function: One connection, one thread. Reads a line, dispatches it into exactly one registry method, writes the reply and logs it. Converts every parse failure, missing field and unknown operations into a response rather than dropping the connection. Acquires no locks.

File: `SubjectRegistry.java`

Responsibility/Function: The five protocol operations, and the only place a subject lock is acquired. Each method takes plain arguments and returns a *ProtocolMessage*. Its nested Store class owns persistence: it writes the full state to a temporary file, `fsync()` it, then renames it over the live file so that a crash can never leave a partial write.

File: `Subject.java`

Responsibility/Function: One subject's data - subject code, capacity, and enrolled student IDs - together with the *ReentrantReadWriteLock* that guards it. Keeping the lock inside the object it protects makes it impossible to reach the data without the lock.

File: `SubjectClient.java`

Responsibility/Function: The admin CLI. Connects once and holds the connection for the session, turns each typed command into a request, sends it, and prints the result.

File: ProtocolMessage.java, ProtocolException.java, SimpleJson.java

Responsibility/Function: Provided scaffold code, used unmodified. They build and parse the JSON request and response messages.

Because `accept()` and `readLine()` on standard input both block, the accept loop runs on a dedicated thread while the console runs on main; this is what allows the console to remain responsive while other clients are being served, and it is also the mechanism by which shutdown works: the console thread closes the listening socket to unblock the acceptor.

Class Diagram and Interaction Diagram (Class diagram + Sequence diagram)

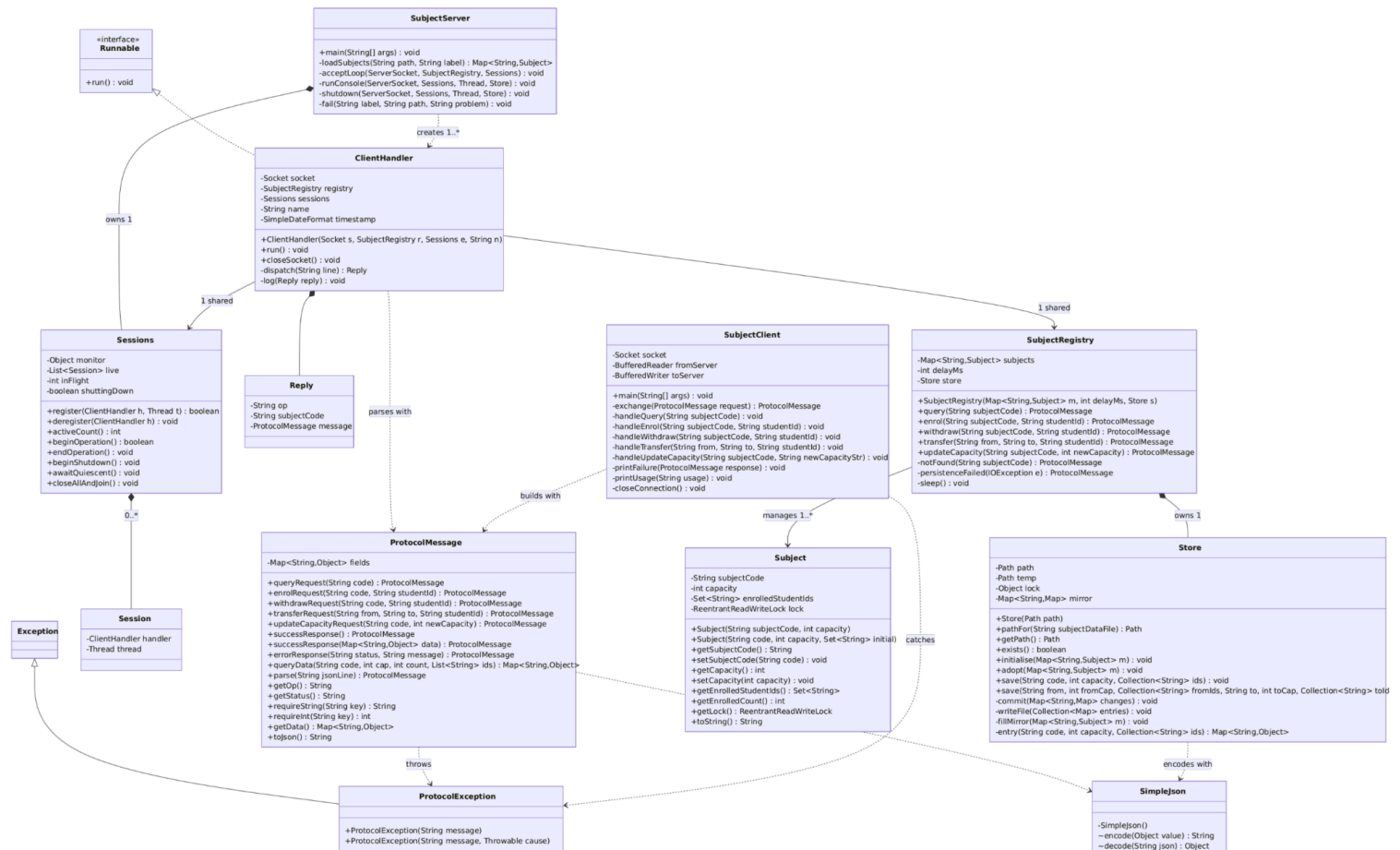


Figure 1. UML Class diagram of the University Subject enrolment system.

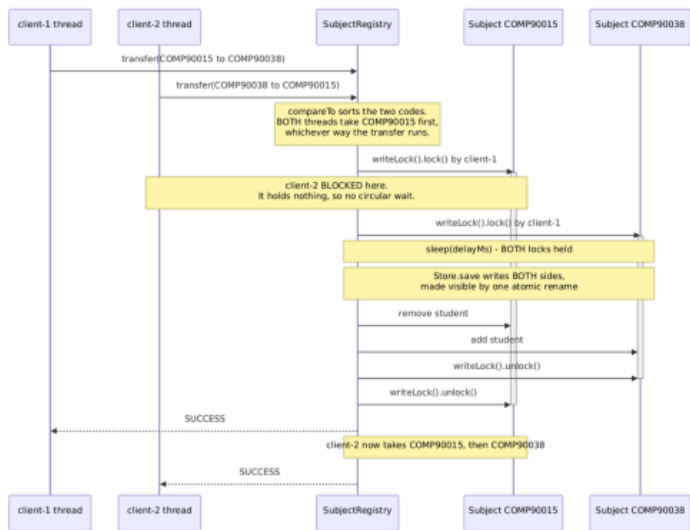


Figure 2. Sequence diagram student transfer scenario

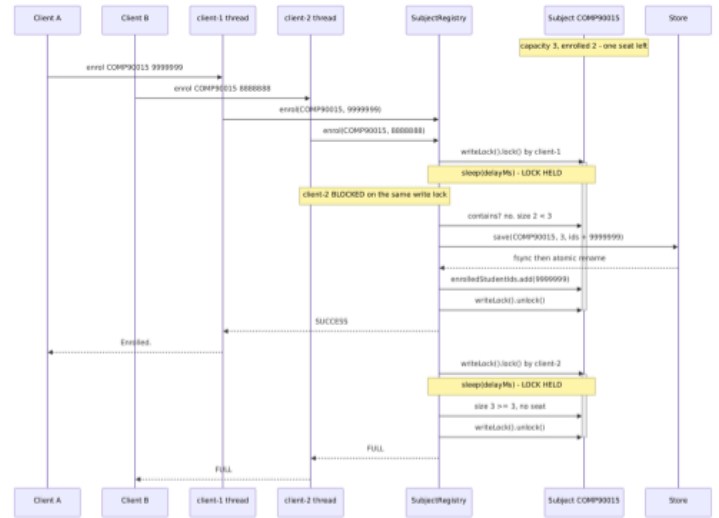


Figure 3. Sequence diagram student enrol scenario

```

> java -jar SubjectServer.jar 8080 subjects-sample.json 5000
Loaded 4 subject(s) from subjects-sample.json
Created persistence file /Users/20jskang/Developer/Distributed-Systems/subjects-sample.state.json
Listening on port 8080 with 4 subject(s), delay 5000ms.
State file: /Users/20jskang/Developer/Distributed-Systems/subjects-sample.state.json
Console commands: status | quit
Client connection number 1 accepted:
  Remote Host: 127.0.0.1
  Remote Port: 56548
  Local Port: 8080
  Named client-1; active connections: 1
Client connection number 2 accepted:
  Remote Host: 127.0.0.1
  Remote Port: 56549
  Local Port: 8080
  Named client-2; active connections: 2
Client connection number 3 accepted:
  Remote Host: 127.0.0.1
  Remote Port: 56550
  Local Port: 8080
  Named client-3; active connections: 3
Client connection number 4 accepted:
  Remote Host: 127.0.0.1
  Remote Port: 56551
  Local Port: 8080
  Named client-4; active connections: 4
[2026-09-10 22:05:32.264] client-1  ENROL          COMP90015          -> SUCCESS
[2026-09-10 22:05:35.689] client-3  ENROL          COMP90038          -> SUCCESS
[2026-09-10 22:05:36.068] client-4  ENROL          COMP90024          -> SUCCESS
[2026-09-10 22:05:37.266] client-2  ENROL          COMP90015          -> FULL
status
Active client connections: 4
quit
Shutting down: no longer accepting new connections.
Waiting for in-flight operations to finish...
Final state is already durable at /Users/20jskang/Developer/Distributed-Systems/subjects-sample.state.json (persisted after every successful write).
Listening socket closed; no longer accepting.
Closing 4 client connection(s).
Connection closed: client-3
Connection closed: client-2
Connection closed: client-1
Connection closed: client-4
Server stopped.
  
```

Figure 4.1. Demo of 4 clients (Last seat race)

```

> java -jar SubjectClient.jar localhost 8080
Connected to localhost:8080.
Type a command: query | enrol | withdraw | transfer | update | quit
enrol COMP90015 555444222111
Enrolled 555444222111 in COMP90015.
update
Usage: update <subjectCode> <newCapacity>
update COMP90015 30
The server closed the connection. Type quit to exit.
  
```

Figure 4.2. Demo Client interface, Closed connection.

Critical Analysis

Data Structures. Subjects are stored in a plain `HashMap<String, Subject>`. Thread safety is achieved through safe publication: the map is fully populated inside `loadSubjects()` prior to thread creation and assigned to a `final` field, and never structurally modified thereafter. This guarantees that worker threads can perform concurrent `get()` operations safely without locking. While the map structure remains `static`, individual `Subject` objects are guarded independently. Inside these subjects, student enrollments use a `LinkedHashSet<String>` secured by a write lock. This guarantees $O(1)$ checks and deterministic query ordering. Finally, the server's own state, the live connection list and the in-flight-operation counter, is a plain `ArrayList` and a plain `int` guarded by a single monitor with `wait()/notifyAll()`.

Persistence format. State is persisted as a single JSON snapshot in the same shape as the subject data file. That makes recovery the existing validated loader pointed at a different path, so the persisted file is re-checked against every file-format rule on each restart rather than trusted. The invariants are hold by constructions: `ENROL` refuses to exceed capacity and `UPDATE_CAPACITY` refuses to fall below the enrolled count, so `capacity >= enrolledStudentIds.length` is always true of what is written.

The difficulty is that rewriting the whole file requires every subject's data, while the writing thread holds the write lock on only the one or two it is changing. Reading the others live would be an unguarded read; acquiring their read locks while holding a write lock would create a cycle, since one thread could hold `W(X)` and want `R(Y)` while another holds `W(Y)` and wants `R(X)`. `Store` resolves this by keeping its own mirror of plain copies: each thread updates only the entry it already holds the write lock for, and no live `Subject` is ever read on the persistence path. Durability comes from writing the mirror to a temporary file, flushing, fsyncing, then renaming it over the live file; a rename is atomic within a filesystem, so a crash leaves either the complete previous file or the complete new one. Persistence happens before the in-memory mutation, so a failed disk write leaves memory untouched and the client is told that the operation did not happen.

Architecture. Because `accept()` and `readLine()` on standard input both block, the accept loop runs on a dedicated thread and the console on main, which is what allows console commands to be served while clients are connected.

Thread model. The server uses thread-per-connection. Each accepted socket is wrapped in a `ClientHandler`, which implements `Runnable` and is started with a raw new `Thread(handler, name)`, named client-N at creation. Together with the acceptor thread and the console on main, a server with N connected clients runs $N + 2$ threads. And this model matches the workload: a client holds one socket and issues a strictly serial request-response loop, so a connection is a session - no dispatch queue, no reordering, and the count that status reports falls out of the session list. It also determines how the shutdown works. A handler parked in a blocking `readLine()` cannot be stopped by polling a flag, so `Sessions` stores each handler with its thread, and shutdown closes the socket to unblock the read before joining.

The cost is that the number of threads follows the number of connections rather than the amount of work, so a connected but idle client still holds a thread, and there is no limit on how many are accepted. A worker pool would fix both, but it adds a queue and breaks the one-thread-per-session link the log depends on.

Concurrent reads, exclusive per-resource writes, and the last-seat race

Each subject holds its own `ReentrantReadWriteLock` in a `final` field, and `SubjectRegistry` is the only class that acquires the lock. `QUERY` takes the read lock, whilst `ENROL`, `WITHDRAW`, `TRANSFER`, and `UPDATE_CAPACITY` take the write lock.

By enforcing this lock design, we allow operations on different subjects to use different lock objects, so that they never block each other. Two `QUERY` operations on the same subject share the read lock and run at the same time. A write holds the write lock, so no other read or write on that subject can start until the operation finishes.

The lock is also stored inside `Subject` rather than in a separate `Map<String, Lock>`. With a separate map, it would be possible to accidentally look up the lock for one subject and then modify a different one. Since `getLock()` is called on the exact object being modified, that mistake cannot happen. Furthermore, the field is deliberately set to `final` so that every thread can see the same lock in use.

The last-seat race condition is prevented by doing the capacity check and the update within a single lock hold. `enrol()` takes the write lock, sleeps, checks whether the student is already enrolled and checks if the class size is still below the capacity, persists the change, and only then adds the student. Because the lock is never released during this process, if two clients race for the last seat, the second client cannot acquire the write lock until the first has finished. By the time the second client gets the lock, the capacity check will fail and return a `FULL` response.

Checking capacity under a read lock, releasing it, and then taking a write lock would introduce a critical gap. Both clients could pass the capacity check before either one performs the update, resulting in an overfilled class. `QUERY` handles this requirement in reverse: the enrolled set is copied into the response while the read lock is still held. Building the response after releasing the lock would mean an unprotected state from a shared reading state.

TRANSFER: Deadlock Prevention and Lock Ordering

Because a `TRANSFER` operation modifies two subjects, it must acquire both write locks simultaneously. If each thread locks its source subject first, two clients attempting transfers in opposite directions would deadlock (Thread 1 holds Subject A waiting for B, while Thread 2 holds Subject B waiting for A).

To eliminate circular wait, lock acquisition follows a deterministic global order. Subject codes are compared using `String.compareTo`, and the lock for the smaller string is always acquired first, regardless of which subject is the source or destination. Because both threads request the locks in the exact same sequence, the first thread acquires the initial lock while the second thread waits without holding any resources, preventing deadlock cycles.

Subject codes serve as the ordering key because of their unique and immutable state. Uniqueness is validated during file loading, guaranteeing a consistent global order across all threads and server restarts.

Both locks are released in reverse order using nested `finally` blocks. State updates for both subjects are made while holding both locks, ensuring that no client ever observes a student in both subjects or neither. For disk persistence, `Store.save()` writes both subject updates to a temporary file and commits them using a single atomic rename, preserving consistency even in the event of a system crash mid-transfer.

The purpose of Artificial Delay argument

The artificial delay exists to make lock contention observable. Without it every operation completes in a fraction of a millisecond on one machine, so an unsynchronised server and a correctly synchronised one would produce an identical output. The delay is a single private helper, `SubjectRegistry.sleep()` under `SubjectRegistry.java`. It wraps `Thread.sleep(delayMs)`, where `delayMs` is the third command-line argument, parsed in `SubjectServer.main()` and passed to the `SubjectRegistry` constructor. In all five of these operations it is called as the first statement inside the `'try'` block that follows lock acquisition, and always before any shared state is read or mutated.

The lock is therefore held for the entire duration of the sleep and released only after the mutation. The only calls to `sleep()` that are not under a lock are the ones where no lock exists to take: a `'subjectCode'` that is not in the map and `TRANSFER`'s same-subject `'INVALID_REQUEST'`. These are resolved from the immutable subject map before any lock is acquired. They still sleep, so response timing stays uniform across every operation, but they hold nothing and block nobody.

Figure 4 shows this in practice with the delay set to 5000ms. `COMP90015` begins at capacity 3 with two students enrolled, so exactly one seat is free, and four clients connect. client-1 returns `SUCCESS` at 22:05:32.264 and client-2 returns `FULL` at 22:05:37.266, one full delay of 5.002 s later: it acquired the write lock only when client-1 released it, slept its own delay while holding it, and only then read the subject as full. The ordering shows this was genuine blocking rather than a late arrival, since client-2 connected second but returned last, behind client-3 and client-4, which connected after it. Those two enrolled into `COMP90038` and `COMP90024` and completed at 22:05:35.689 and 22:05:36.068 while client-2 was still waiting, because a different subject meant a different lock. Had the sleep been placed outside the lock hold, all four operations would have finished inside the same 5s window.

Because the lock is held across the whole check-persist-update sequence, that sequence had to fit inside one uninterrupted lock hold, which is the same property that closes the last-seat race. It also forced the shutdown design: an operation can be asleep holding a lock when `quit` is typed, so `SubjectServer.shutdown()` cannot simply close sockets (it marks the shutdown, closes the listening socket, then blocks in `Sessions.awaitQuiescent()` until the in-flight count reaches zero).